

Anexo

Configuración técnica y parametrización de la PoC

Diseño y validación de una plataforma segura de APIs para integración bancaria

Autor	Antonio Martos Gavilán
Titulación	Grado de Ingeniería Informática
Área	Sistemas de Gestión del Conocimiento
Tutor	Oscar Martín Salas
Fecha	08/06/2026
Versión	1.2 - Documento de configuración final para entrega

Índice del anexo

1. Finalidad del anexo
2. Alcance de la configuración documentada
3. Principios de configuración segura
4. Inventario de ficheros de configuración
5. Configuración de Microsoft Entra ID
6. Configuración de Docker Compose
7. Configuración de Kong Gateway
8. Configuración de las APIs .NET
9. Configuración de RabbitMQ y del consumidor asíncrono
10. Configuración de Prometheus y Grafana
11. Configuración de seguridad y *hardening* básico
12. Scripts auxiliares de configuración y validación
13. Verificación de configuración
14. Incidencias de configuración y resolución
15. Checklist de evidencias de configuración
16. Criterios de aceptación de la configuración

- 17. Matriz de trazabilidad configuración-requisito-evidencia
- 18. Limitaciones y consideraciones de seguridad para entorno real
- 19. Cierre del anexo
- 20. Referencias bibliográficas

1. Finalidad del anexo

Este anexo documenta la configuración técnica de la prueba de concepto (PoC) desarrollada en el TFG. Su finalidad es permitir que la persona evaluadora comprenda qué parámetros, registros, rutas, roles, *plugins*, variables y ficheros hacen posible la ejecución segura del entorno descrito en el Anexo T de instalación.

A diferencia del anexo de instalación, que describe el procedimiento operativo para levantar la PoC, este documento se centra en la parametrización de sus componentes: Microsoft Entra ID, Kong Gateway, APIs .NET, RabbitMQ, Prometheus, Grafana y ficheros auxiliares. La configuración se presenta de forma reproducible, pero sin exponer secretos reales ni valores sensibles.

Criterio de seguridad documental: todas las credenciales, secretos, tokens, identificadores privados o claves completas se sustituyen por marcadores como <TENANT_ID>, <CLIENT_ID>, <CLIENT_SECRET>, <RESOURCE_APP_ID> o <RSA_PUBLIC_KEY>. Los valores reales solo deben existir en el fichero .env local del laboratorio.

2. Alcance de la configuración documentada

La configuración descrita corresponde al entorno local de laboratorio desplegado mediante Docker Compose. No se documenta una configuración productiva con alta disponibilidad, WAF, rotación automática de secretos, despliegue en AKS ni gestión corporativa de identidades. Estas capacidades quedan identificadas como líneas de evolución futura.

Dominio	Componente	Configuración cubierta
Identidad	Microsoft Entra ID	Registro de aplicaciones, app roles, <i>client credentials</i> y emisión de JWT RS256.
Gateway	Kong Gateway CE	Servicios, rutas, <i>consumers</i> , plugin JWT, <i>rate limiting</i> y plugin Prometheus.
Backends	APIs .NET 8	Variables de ejecución, audiencia, roles requeridos y conectividad con RabbitMQ.
Mensajería	RabbitMQ	Usuario local, cola cliente-creado, persistencia de mensajes y consumo por Worker Service.
Observabilidad	Prometheus y Grafana	<i>Scraping</i> de métricas Kong, <i>datasource</i> Prometheus y <i>dashboard</i> operativo.
Seguridad	Configuración local	Gestión de .env, no versionado de secretos y exposición controlada de puertos.

Tabla U.1. Alcance de la configuración documentada

3. Principios de configuración segura

La parametrización de la PoC sigue un conjunto de principios operativos orientados a reproducibilidad, seguridad y claridad académica.

Código	Principio	Aplicación en la PoC
PCS-01	Separación código/configuración	El código fuente no contiene secretos ni parámetros privados. La configuración local se inyecta mediante .env y variables de entorno.
PCS-02	Mínimo privilegio	Los consumidores solo reciben los app roles necesarios para sus pruebas: cliente completo y cliente limitado.
PCS-03	Validación en dos capas	Kong valida autenticidad del JWT, los backends verifican roles de aplicación antes de ejecutar operaciones.

Código	Principio	Aplicación en la PoC
PCS-04	Reproducibilidad	La configuración se puede recrear desde scripts o llamadas documentadas a la Admin API de Kong.
PCS-05	Observabilidad desde el diseño	El plugin Prometheus de Kong se activa como parte de la configuración base, no como añadido posterior.
PCS-06	No exposición de secretos	Los valores sensibles se ocultan en anexos, capturas y repositorio Git.

Tabla U.2. Principios de configuración segura

4. Inventario de ficheros de configuración

La configuración de la PoC se distribuye en varios ficheros del repositorio. La Tabla U.3 recoge el inventario esperado y su función.

Fichero	Propósito
docker-compose.yml	Define servicios, redes, volúmenes, puertos y variables de entorno de los contenedores.
env.example.txt / .env.example	Plantilla segura de variables sin valores sensibles reales.
.env	Fichero local con valores reales de laboratorio. Debe estar excluido del repositorio.
get_token.sh	Script auxiliar para solicitar tokens OAuth 2.0 client credentials a Entra ID.
scripts/configure_kong.sh	Script recomendado para declarar servicios, rutas, plugins y consumers en Kong.
prometheus/prometheus.yml	Configura el scraping del endpoint de métricas de Kong.
grafana/provisioning/datasources/	Datasource Prometheus para Grafana, si se usa provisioning.
grafana/provisioning/dashboards/	Dashboard de la PoC o referencia al JSON exportado desde Grafana.
appsettings.json	Configuración no sensible de las APIs .NET cuando no se inyecta por variables de entorno.

Tabla U.3. Inventario de ficheros de configuración

5. Configuración de Microsoft Entra ID

Microsoft Entra ID actúa como proveedor de identidad y servidor de autorización. La PoC utiliza el flujo OAuth 2.0 *client credentials*, adecuado para integración máquina a máquina entre plataformas. Entra ID emite tokens JWT firmados con RS256 que posteriormente se validan en Kong Gateway.

5.1. Registros de aplicación

Tipo	Nombre lógico	Función
Aplicación recurso	tfg-api-resource	Representa la API protegida. Define Application ID URI y app roles.
Aplicación cliente principal	tfg-api-client	Cliente machine-to-machine con roles de lectura/escritura de clientes y lectura de cuentas.

Tipo	Nombre lógico	Función
Aplicación cliente limitada	tfg-api-client-clientes	Cliente con permiso limitado para demostrar autorización diferencial.

Tabla U.4. Registros de aplicación de Entra ID

5.2. App roles definidos

Nombre	Valor claim	Operaciones	Tipo
Cientes.Read.App	Cientes.Read.App	GET /api/v1/clientes y GET /api/v1/clientes/{id}	Solo lectura
Cientes.Write.App	Cientes.Write.App	POST /api/v1/clientes	Escritura
Cuentas.Read.App	Cuentas.Read.App	GET /api/v1/cuentas, GET /api/v1/cuentas/{id}, GET /api/v1/cuentas/{id}/saldo	Solo lectura

Tabla U.5. App roles usados por la PoC

5.3. Parámetros de emisión de token

Parámetro	Valor / patrón	Uso
Tenant	<TENANT_ID>	Identificador del <i>tenant</i> de Entra ID.
Endpoint token	https://login.microsoftonline.com/<TENANT_ID>/oauth2/v2.0/token	Endpoint OAuth 2.0 v2.
Grant type	client_credentials	Flujo machine-to-machine sin usuario final.
Scope	api://<RESOURCE_APP_ID>/.default	Solicita los permisos de aplicación concedidos.
Audiencia	api://<RESOURCE_APP_ID>	Valor esperado de aud en el JWT.
Algoritmo	RS256	Firma asimétrica validable mediante clave pública.
Claims relevantes	iss, aud, appid, roles, exp, nbf	Claims usados para validación y autorización.

Tabla U.6. Parámetros de emisión de token

5.4. Plantilla de variables Entra ID

```
# .env - variables relacionadas con Microsoft Entra ID
ENTRA_TENANT_ID=<TENANT_ID>
ENTRA_RESOURCE_APP_ID=<RESOURCE_APP_ID>
ENTRA_CLIENT_ID=<CLIENT_ID>
ENTRA_CLIENT_SECRET=<CLIENT_SECRET>
ENTRA_SCOPE=api://<RESOURCE_APP_ID>/.default
ENTRA_TOKEN_ENDPOINT=https://login.microsoftonline.com/<TENANT_ID>/oauth2/v2.0/token
```

Control documental: no capturar ni entregar el valor real de `ENTRA_CLIENT_SECRET`. En evidencias se debe mostrar solo que la variable existe, ocultando su contenido.

6. Configuración de Docker Compose

Docker Compose actúa como mecanismo de orquestación local de la PoC. El fichero `docker-compose.yml` debe declarar los servicios funcionales, sus redes, volúmenes, puertos y dependencias de arranque.

Servicio	Imagen / tecnología	Responsabilidad	Puertos
kong-database	postgres:15	Almacén de configuración de Kong	5432 interno
kong-migrations	kong:3.6	Ejecución de migraciones iniciales	Sin puerto público
kong	kong:3.6	API Gateway, Admin API y métricas	8000, 8001, 8100
clientes-api	.NET 8	<i>Backend</i> de clientes	8081 -> 80
cuentas-api	.NET 8	<i>Backend</i> de cuentas	8082 -> 80
rabbitmq	rabbitmq:3-management	Broker AMQP y consola	5672, 15672
cliente-creado-consumer	.NET 8 Worker	Consumidor de eventos	Sin puerto HTTP
prometheus	prom/prometheus	<i>Scraping</i> y almacenamiento de métricas	9090
grafana	grafana/grafana	Visualización de métricas	3000

Tabla U.7. Servicios configurados en Docker Compose

6.1. Variables principales de Kong en Compose

```
# Fragmento conceptual del servicio kong
KONG_DATABASE=postgres
KONG_PG_HOST=kong-database
KONG_PROXY_ACCESS_LOG=/dev/stdout
KONG_ADMIN_ACCESS_LOG=/dev/stdout
KONG_PROXY_ERROR_LOG=/dev/stderr
KONG_ADMIN_ERROR_LOG=/dev/stderr
KONG_ADMIN_LISTEN=0.0.0.0:8001
KONG_STATUS_LISTEN=0.0.0.0:8100
KONG_PROXY_LISTEN=0.0.0.0:8000
```

Estas variables permiten diferenciar el plano de datos (8000), la Admin API local (8001) y el endpoint de estado/métricas (8100).

6.2. Redes y volúmenes

Elemento	Uso
Red Docker interna	Permite resolver upstreams por nombre de servicio, por ejemplo <code>http://clientes-api:80</code> .
Volumen PostgreSQL	Conserva la configuración de Kong mientras no se ejecute <code>docker compose down -v</code> .
Volumen RabbitMQ	Mantiene estado del <i>broker</i> y cola durante reinicios simples.
Volumen Prometheus	Conserva la TSDB de métricas durante pruebas.
Volumen Grafana	Conserva <i>datasources</i> y <i>dashboards</i> si no se usa <i>provisioning</i> .

Tabla U.8. Redes y volúmenes de configuración

7. Configuración de Kong Gateway

Kong Gateway es el punto único de exposición de las APIs. Su configuración se realiza a través de la Admin API en `http://localhost:8001` o mediante scripts auxiliares. La configuración mínima incluye servicios, rutas, *plugins* de seguridad, plugin de observabilidad, *consumers* y credenciales JWT.

7.1. Servicios y rutas

Servicio	Upstream	Ruta	Path público	strip_path
clientes-service	http://clientes-api:80	clientes-route	/clientes	false
cuentas-service	http://cuentas-api:80	cuentas-route	/cuentas	true

Tabla U.9. Servicios y rutas configurados en Kong

```
# Servicios
curl -s -X POST http://localhost:8001/services \
  --data name=clientes-service \
  --data url=http://clientes-api:80

curl -s -X POST http://localhost:8001/services \
  --data name=cuentas-service \
  --data url=http://cuentas-api:80

# clientes-route con strip_path=false (RMQ-01) y cuentas-route con strip_path=true
curl -s -X POST http://localhost:8001/services/clientes-service/routes \
  --data name=clientes-route \
  --data 'paths[]=/clientes' \
  --data strip_path=false

curl -s -X POST http://localhost:8001/services/cuentas-service/routes \
  --data name=cuentas-route \
  --data 'paths[]=/cuentas' \
  --data strip_path=true
```

7.2. Plugin JWT

El plugin JWT se activa sobre los servicios protegidos. Su finalidad es validar tokens firmados por Entra ID, asociar el token entrante con un consumer de Kong mediante la claim appid y propagar el JWT al backend para que las APIs .NET puedan evaluar la *claim* roles.

Parámetro	Valor	Finalidad
Plugin	jwt	Autenticación centralizada de peticiones.
Ámbito	clientes-service y cuentas-service	Ambas APIs quedan protegidas.
key_claim_name	appid	Permite usar el client_id de Entra ID como identificador de <i>consumer</i> .
algorithm	RS256	Validación mediante clave pública RSA.
claims_to_verify	exp, nbf	Verificación de vigencia temporal del token.
hide_credentials	false	Propaga el JWT al backend para evaluar roles.

Tabla U.10. Parámetros del plugin JWT

```
# Activación del plugin JWT en servicios protegidos
curl -s -X POST http://localhost:8001/services/clientes-service/plugins \
  --data name=jwt \
  --data config.key_claim_name=appid \
  --data config.claims_to_verify=exp \
  --data config.claims_to_verify=nbf \
  --data config.hide_credentials=false
```



```
curl -s -X POST http://localhost:8001/services/cuentas-service/plugins \
  --data name=jwt \
  --data config.key_claim_name=appid \
  --data config.claims_to_verify=exp \
  --data config.claims_to_verify=nbfs \
  --data config.hide_credentials=false
```

7.3. Consumer y credencial JWT RS256

Cada aplicación cliente de Entra ID que consuma la API debe representarse como *consumer* en Kong. La clave del *consumer* debe coincidir con el appid incluido en el JWT emitido por Entra ID.

```
# Crear consumer asociado al client_id de Entra ID
curl -s -X POST http://localhost:8001/consumers \
  --data username=tfg-api-client \
  --data custom_id=<CLIENT_ID>

# Registrar credencial JWT con algoritmo RS256
curl -s -X POST http://localhost:8001/consumers/tfg-api-client/jwt \
  --data key=<CLIENT_ID> \
  --data algorithm=RS256 \
  --data rsa_public_key='<RSA_PUBLIC_KEY_FROM_ENTRA_JWKS>'
```

Aspecto crítico: la clave pública debe corresponder al kid usado por Entra ID para firmar el token. Si Entra ID rota la clave, debe actualizarse la credencial JWT en Kong.

7.4. Rate limiting

El *rate limiting* se configura para demostrar control de tráfico y mitigación de consumo irrestricto de recursos. En la PoC se emplea un umbral bajo para facilitar la validación académica.

```
# Umbral de laboratorio: 10 peticiones por minuto por consumer
curl -s -X POST http://localhost:8001/services/clientes-service/plugins \
  --data name=rate-limiting \
  --data config.minute=10 \
  --data config.policy=local \
  --data config.limit_by=consumer

curl -s -X POST http://localhost:8001/services/cuentas-service/plugins \
  --data name=rate-limiting \
  --data config.minute=10 \
  --data config.policy=local \
  --data config.limit_by=consumer
```

7.5. Plugin Prometheus

```
# Plugin global de métricas para Prometheus
curl -s -X POST http://localhost:8001/plugins \
  --data name=prometheus \
  --data config.status_code_metrics=true \
  --data config.latency_metrics=true \
  --data config.bandwidth_metrics=true
```

```
# Verificación del endpoint de métricas
curl -s http://localhost:8100/metrics | head -20
```

8. Configuración de las APIs .NET

Las APIs de Clientes y Cuentas se ejecutan como contenedores ASP.NET Core 8. Kong valida el token antes de reenviar la petición, pero los *backends* mantienen una segunda capa de autorización mediante evaluación de la *claim* roles.

Variable	Valor / patrón	Uso
ASPNETCORE_URLS	http://+:80	Expone la API dentro del contenedor en el puerto 80.
Authentication__Audience	api://<RESOURCE_APP_ID>	Audiencia esperada del token.
Authentication__Authority	https://login.microsoftonline.com/<TENANT_ID>/v2.0	Emisor de tokens.
Authorization__RoleClaim	roles	Claim usada para autorización.
RabbitMq__Host	rabbitmq	Host interno del <i>broker</i> para ClientesApi.
RabbitMq__Queue	cliente-creado	Cola usada para eventos de alta de cliente.

Tabla U.11. Variables de ejecución de las APIs .NET

8.1. Mapeo de roles por endpoint

Método	Endpoint	Rol requerido	Finalidad
GET	/api/v1/clientes	Clientes.Read.App	Listado de clientes.
GET	/api/v1/clientes/{id}	Clientes.Read.App	Consulta de cliente por identificador.
POST	/api/v1/clientes	Clientes.Write.App	Alta de cliente y publicación de evento.
GET	/api/v1/cuentas	Cuentas.Read.App	Listado de cuentas.
GET	/api/v1/cuentas/{id}	Cuentas.Read.App	Consulta de cuenta.
GET	/api/v1/cuentas/{id}/saldo	Cuentas.Read.App	Consulta de saldo.

Tabla U.12. Autorización diferencial en backends .NET

9. Configuración de RabbitMQ y del consumidor asíncrono

RabbitMQ se incorpora para demostrar integración asíncrona básica mediante el evento ClienteCreado. La configuración se mantiene intencionadamente sencilla para no desplazar el foco principal del TFG: plataforma segura de APIs.

Parámetro	Valor / patrón	Uso
RABBITMQ_DEFAULT_USER	<RABBITMQ_USER>	Usuario local de laboratorio.
RABBITMQ_DEFAULT_PASS	<RABBITMQ_PASSWORD>	Contraseña local de laboratorio.
Host AMQP	rabbitmq	Nombre de servicio resoluble dentro de Docker.
Puerto AMQP	5672	Canal de publicación y consumo.
Puerto Management	15672	Interfaz web de administración.

Parámetro	Valor / patrón	Uso
Cola	cliente-creado	Cola persistente para eventos de alta de cliente.
Ack	BasicAck	Confirmación tras procesamiento correcto por el Worker.

Tabla U.13. Configuración de RabbitMQ

```
# Variables orientativas para API y Worker
RabbitMq__Host=rabbitmq
RabbitMq__Port=5672
RabbitMq__User=<RABBITMQ_USER>
RabbitMq__Password=<RABBITMQ_PASSWORD>
RabbitMq__Queue=cliente-creado
```

10. Configuración de Prometheus y Grafana

10.1. Prometheus

Prometheus recoge métricas desde el endpoint de estado de Kong. El scraping se configura con un intervalo corto para facilitar la validación durante las pruebas. La configuración final usa `kong_http_requests_total` para la distribución de códigos HTTP, métrica validada en la PoC y coherente con el dashboard definitivo de Grafana.

```
# prometheus/prometheus.yml
scrape_configs:
  - job_name: 'kong'
    scrape_interval: 15s
    static_configs:
      - targets: ['kong:8100']
```

Métrica / consulta	Uso en la PoC
<code>kong_nginx_requests_total</code>	Volumen total de peticiones observadas por Kong.
<code>kong_latency_ms_sum / kong_latency_ms_count</code>	Base para calcular latencia media del gateway y de servicios proxied.
<code>kong_http_requests_total</code>	Distribución de respuestas por código HTTP validada en el dashboard final de Grafana.
<code>kong_datastore_reachable</code>	Estado del datastore de Kong: 1 operativo, 0 no alcanzable.
<code>up{job='kong'}</code>	Estado del scraping de Prometheus hacia Kong.

Tabla U.14. Métricas relevantes de Prometheus

10.2. Grafana

Grafana visualiza las métricas recogidas por Prometheus. El *datasource* debe apuntar al nombre interno del servicio Prometheus, no a localhost, porque Grafana se ejecuta dentro de la red Docker.

```
# Datasource Prometheus para provisioning de Grafana
apiVersion: 1

datasources:
  - name: Prometheus
```

```

type: prometheus
access: proxy
url: http://prometheus:9090
isDefault: true

```

Panel	Consulta base	Interpretación
Kong - Tasa de peticiones	rate(kong_nginx_requests_total[1m])	Volumen de peticiones por segundo.
Kong - Respuestas HTTP por código	sum by (code)(rate(kong_http_requests_total[5m]))	Evolución temporal de respuestas 2xx, 4xx y 5xx.
Kong - Total de respuestas por código	sum by (code)(increase(kong_http_requests_total[\$__range]))	Conteo absoluto por código HTTP en el rango seleccionado.
Kong - Errores técnicos 5xx	sum(rate(kong_http_requests_total{code=~"5.."}[5m]))	Errores técnicos reales de la plataforma.
Kong - Peticiones rechazadas o limitadas	sum(increase(kong_http_requests_total{code=~"401 403 429"}[\$__range]))	Rechazos de seguridad o gobierno de tráfico.
Kong - Estado del datastore	kong_datastore_reachable	Salud del datastore de Kong: 1 operativo, 0 no alcanzable.

Tabla U.15. Paneles esperados en Grafana

11. Configuración de seguridad y hardening básico

Control	Configuración	Justificación
.env no versionado	El fichero .env debe estar en .gitignore.	Evita filtración de <i>client secrets</i> y contraseñas.
Admin API local	Kong Admin API expuesta solo en laboratorio.	En producción no debe exponerse públicamente.
JWT RS256	Firma asimétrica validada con clave pública.	Evita secretos compartidos entre Entra ID y <i>gateway</i> .
Claims exp y nbf	Verificación de vigencia temporal.	Mitiga reutilización de tokens fuera de ventana válida.
Roles de aplicación	Autorización por operación en <i>backend</i> .	Reduce privilegios y demuestra autorización diferencial.
Rate limiting	Umbral de laboratorio por minuto.	Mitiga consumo irrestricto de recursos.
Datos simulados	No se usan datos reales de clientes.	Reduce impacto legal y ético de la PoC.

Tabla U.16. Controles de seguridad configurados

12. Scripts auxiliares de configuración y validación

La PoC puede apoyarse en scripts para evitar errores manuales. Los nombres concretos pueden variar en el repositorio final, pero se recomienda mantener las responsabilidades descritas en la Tabla U.17.

Script	Responsabilidad
get_token.sh	Solicita token a Entra ID usando <i>client credentials</i> y variables <i>.env</i> .
configure_kong.sh	Crea servicios, rutas, <i>plugins</i> , <i>consumers</i> y credenciales JWT.
verify_kong.sh	Consulta <i>/services</i> , <i>/routes</i> , <i>/plugins</i> y <i>/consumers</i> para confirmar configuración.
test_functional.sh	Ejecuta llamadas 200, 401, 403 y 429 contra Kong.
benchmark_gateway_overhead.sh	Opcional: compara latencia directa al <i>backend</i> frente a latencia vía <i>gateway</i> .

Tabla U.17. Scripts auxiliares recomendados

```
# Permisos de ejecución recomendados
chmod +x get_token.sh
chmod +x scripts/*.sh

# Ejecución orientativa
./get_token.sh
./scripts/configure_kong.sh
./scripts/verify_kong.sh
```

13. Verificación de configuración

Los siguientes comandos permiten verificar que la configuración queda registrada correctamente antes de ejecutar las pruebas funcionales completas.

```
# Docker Compose
docker compose config --quiet
docker compose ps

# Kong: servicios, rutas y plugins
curl -s http://localhost:8001/services | jq '.data[].name'
curl -s http://localhost:8001/routes | jq '.data[] | {name, paths, strip_path}'
curl -s http://localhost:8001/plugins | jq '.data[] | {name, enabled}'
curl -s http://localhost:8001/consumers | jq '.data[] | {username, custom_id}'

# Entra ID: token recibido y claims básicos, sin mostrar token completo
TOKEN=$(./get_token.sh | tail -1)
printf "Token recibido: %.25s...\n" "$TOKEN"

# Prometheus y Grafana
curl -s http://localhost:9090/-/ready
curl -s http://localhost:8100/metrics | head -20
```

14. Incidencias de configuración y resolución

Incidencia	Causa probable	Resolución
401 con token válido	Kong no encuentra <i>consumer</i> por appid o clave RS256 incorrecta.	Verificar <i>key_claim_name=appid</i> , <i>consumer custom_id</i> y <i>rsa_public_key</i> .
403 inesperado	El token no incluye la <i>claim roles</i> o falta <i>app role</i> requerido.	Revisar asignación de <i>app roles</i> y consentimiento de administrador en <i>Entra ID</i> .
404 en rutas	<i>strip_path</i> incorrecto o <i>path</i> público no coincide con <i>backend</i> .	Comprobar <i>strip_path</i> : <i>clientes-route</i> en <i>false</i> (RMQ-01) y <i>cuentas-route</i> en <i>true</i> .
Prometheus target DOWN	Prometheus apunta a <i>localhost</i> en lugar de <i>kong:8100</i> .	Usar <i>targets</i> : ['kong:8100'] dentro de <i>Docker Compose</i> .
Grafana sin conexión	<i>Datasource</i> apunta a <i>localhost:9090</i> desde el contenedor.	Configurar <i>datasource</i> con url <i>http://prometheus:9090</i> .
RabbitMQ auth failed	Usuario/contraseña no coinciden entre <i>broker</i> , <i>API</i> y <i>Worker</i> .	Unificar variables <i>RabbitMq__User</i> y <i>RabbitMq__Password</i> en <i>.env</i> .
Kong sin plugins	<i>configure_kong.sh</i> no se ejecutó o <i>PostgreSQL</i> se reinició.	Repetir script de configuración tras <i>docker compose up -d</i> .
Rate limiting no dispara	Plugin aplicado en ruta incorrecta o umbral demasiado alto.	Consultar <i>/plugins</i> y generar más de 10 peticiones/minuto.

Tabla U.18. Incidencias habituales de configuración

15. Checklist de evidencias de configuración

Código	Evidencia	Contenido mínimo
E-CFG-01	Plantilla <i>.env</i>	Captura de variables con secretos ocultos.
E-CFG-02	Registros <i>Entra ID</i>	Aplicación recurso y aplicaciones cliente creadas.
E-CFG-03	<i>App roles</i>	<i>Clientes.Read.App</i> , <i>Clientes.Write.App</i> y <i>Cuentas.Read.App</i> habilitados.
E-CFG-04	Asignación de roles	Cliente principal y cliente limitado con roles correctos.
E-CFG-05	<i>docker compose config</i>	Configuración validada sin errores de sintaxis.
E-CFG-06	<i>Kong services/routes</i>	<i>Admin API</i> muestra <i>clientes-service</i> y <i>cuentas-service</i> .
E-CFG-07	<i>Kong plugins</i>	<i>jwt</i> , <i>rate-limiting</i> y <i>prometheus</i> activos.
E-CFG-08	<i>Consumer JWT</i>	<i>Consumer</i> asociado al <i>client_id</i> y credencial RS256 registrada.
E-CFG-09	<i>APIs .NET</i>	Variables de audiencia, autoridad y roles aplicadas.
E-CFG-10	<i>RabbitMQ</i>	Cola cliente-creado y credenciales coherentes.
E-CFG-11	<i>Prometheus</i>	Target <i>kong:8100</i> en estado <i>UP</i> .
E-CFG-12	<i>Grafana</i>	<i>Datasource Prometheus</i> probado correctamente.
E-CFG-13	Prueba 403	Cliente limitado sin <i>Cuentas.Read.App</i> recibe 403 en <i>/cuentas</i> .
E-CFG-14	Prueba 429	<i>Rate limiting</i> devuelve 429 y <i>Retry-After</i> .

Tabla U.19. Checklist final de evidencias de configuración

16. Criterios de aceptación de la configuración

Para que la configuración descrita en este anexo pueda considerarse válida en el contexto de la PoC, no basta con que los ficheros existan o que los contenedores arranquen. Es necesario que la parametrización

permita demostrar, de forma observable y repetible, autenticación, autorización, gobierno de tráfico, observabilidad, segregación de secretos y coherencia entre componentes. Los criterios siguientes convierten la configuración en una evidencia evaluable y conectan el anexo con la validación funcional y de seguridad de la memoria.

Código	Criterio de aceptación	Configuración que lo acredita	Evidencia esperada
CA-CFG-01	La configuración es reproducible desde el repositorio entregado.	docker-compose.yml, .env.example y scripts auxiliares documentados.	docker compose config --quiet sin errores y servicios definidos de forma coherente.
CA-CFG-02	La identidad federada queda correctamente parametrizada.	Registros de aplicación, app roles y flujo client credentials en Microsoft Entra ID.	Token JWT emitido con aud, iss, appid, exp, nbf y roles esperados.
CA-CFG-03	El gateway gobierna todo el acceso funcional a las APIs.	Servicios, rutas, consumers, credencial RS256, plugin JWT y rate limiting en Kong.	Admin API muestra servicios, rutas y plugins activos, las llamadas válidas pasan por el puerto 8000.
CA-CFG-04	La autorización diferencial queda demostrada.	Roles Clientes.Read.App, Clientes.Write.App y Cuentas.Read.App evaluados por las APIs .NET.	El cliente limitado accede a clientes y recibe 403 Forbidden al invocar cuentas.
CA-CFG-05	La observabilidad queda integrada desde la configuración.	Plugin Prometheus de Kong, prometheus.yml y datasource Prometheus en Grafana.	Target kong:8100 en estado UP y paneles con tráfico, latencia y códigos HTTP.
CA-CFG-06	Los secretos no quedan expuestos en documentación ni repositorio.	.env excluido, .env.example sin valores reales y capturas con valores ocultos.	No aparecen client secrets, contraseñas, tokens completos ni claves privadas en los entregables.
CA-CFG-07	La mensajería asíncrona se valida sin desplazar el foco principal del TFG.	RabbitMQ, cola cliente-creado y Worker Service .NET configurados como extensión parcial.	Alta de cliente genera evento y el consumidor registra su recepción o confirmación.

Tabla U.20. Criterios de aceptación de la configuración

17. Matriz de trazabilidad configuración-requisito-evidencia

La Tabla U.21 relaciona los principales bloques de configuración con los requisitos y evidencias que permiten defender que la PoC no es solo un despliegue técnico, sino una arquitectura de integración segura, gobernada y observable. Esta trazabilidad facilita al tribunal verificar qué parte de la configuración contribuye a cada objetivo del TFG.

Bloque de configuración	Requisito o propiedad cubierta	Evidencia vinculada	Riesgo mitigado
Microsoft Entra ID y app roles	Autenticación federada y autorización por permisos de aplicación.	E-CFG-02, E-CFG-03, E-CFG-04 y token con roles.	Uso de credenciales no federadas o privilegios excesivos.

Bloque de configuración	Requisito o propiedad cubierta	Evidencia vinculada	Riesgo mitigado
Kong Gateway: servicios, rutas y plugin JWT	Gobierno centralizado del acceso y validación criptográfica de tokens.	E-CFG-06, E-CFG-07 y E-CFG-08.	Exposición directa de <i>backends</i> o aceptación de tokens inválidos.
Rate limiting en Kong	Control de tráfico y protección básica frente a abuso de consumo.	E-CFG-14 y respuesta 429 con Retry-After.	Consumo irrestricto de recursos de la PoC.
APIs .NET con validación de roles	Defensa en profundidad y autorización por operación.	E-CFG-09 y prueba 403 del cliente limitado.	Acceso a operaciones no autorizadas si el <i>gateway</i> se configurase de forma incompleta.
Prometheus y Grafana	Observabilidad técnica de tráfico, salud y errores.	E-CFG-11 y E-CFG-12.	Falta de visibilidad sobre disponibilidad, errores y comportamiento del <i>gateway</i> .
RabbitMQ y consumidor asíncrono	Desacoplamiento parcial mediante eventos de negocio.	E-CFG-10 y logs del Worker Service.	Acoplamiento síncrono total entre dominios.
.env, .gitignore y plantillas seguras	Separación entre código, configuración y secretos.	E-CFG-01 y revisión del repositorio.	Filtración accidental de secretos en documentación o control de versiones.

Tabla U.21. Trazabilidad configuración-requisito-evidencia

18. Limitaciones y consideraciones de seguridad para entorno real

La configuración documentada es suficiente para validar académicamente la arquitectura en un entorno local controlado, pero no debe interpretarse como una configuración productiva. La Tabla U.22 explicita las principales limitaciones y la evolución razonable que sería necesaria en un entorno bancario real. Esta separación es relevante para evitar una sobreafirmación del alcance y para mantener la coherencia con la naturaleza de PoC del trabajo.

Aspecto	Tratamiento en la PoC	Evolución recomendada en entorno real
Kong Admin API	Expuesta localmente para configuración y verificación.	Restringir a red privada de administración, aplicar autenticación, RBAC, mTLS y auditoría.
Gestión de secretos	Uso de .env local y valores ocultos en documentación.	Usar un gestor de secretos corporativo, rotación automática, segregación de permisos y trazabilidad de acceso.
Validación JWT	Registro manual de clave pública RS256 en Kong CE.	Automatizar sincronización con JWKS, gestionar rotación de claves y evaluar OIDC nativo o API Management corporativo.

Aspecto	Tratamiento en la PoC	Evolución recomendada en entorno real
Disponibilidad	Docker Compose local sin alta disponibilidad.	Despliegue en plataforma gestionada, configuración declarativa, <i>backups</i> , escalado y tolerancia a fallos.
RabbitMQ	Broker local con cola demostrativa.	Configurar TLS, usuarios por aplicación, colas durables/quorum, monitorización y políticas de reintento.
Observabilidad	Métricas locales de gateway y dashboard Grafana.	Añadir alertas, retención, trazas distribuidas, logs centralizados y gobierno de acceso a <i>dashboards</i> .
Datos de prueba	Datos simulados sin información personal real.	Aplicar minimización, anonimización, control de acceso, evaluación de impacto y cumplimiento normativo si se usaran datos reales.

Tabla U.22. Limitaciones y evolución de la configuración hacia un entorno real

19. Cierre del anexo

La configuración técnica queda documentada de forma reproducible, segura y alineada con los objetivos del TFG. Si se completan las evidencias E-CFG-01 a E-CFG-14 y los criterios CA-CFG-01 a CA-CFG-07, la PoC dispone de una base suficiente para demostrar autenticación federada, autorización por roles, gobierno centralizado mediante gateway, observabilidad operativa y extensión parcial mediante mensajería asíncrona. El documento mantiene explícitamente la separación entre configuración académica de laboratorio y configuración productiva, evitando presentar como implantación real aquello que forma parte de una validación controlada.

20. Referencias bibliográficas

- Docker Inc. (s. f.). Compose file reference. Docker Docs. <https://docs.docker.com/reference/compose-file/>
- Grafana Labs. (s. f.). Configure the Prometheus data source. Grafana documentation. <https://grafana.com/docs/grafana/latest/datasources/prometheus/configure/>
- Grafana Labs. (s. f.). Provision Grafana. Grafana documentation. <https://grafana.com/docs/grafana/latest/administration/provisioning/>
- Kong Inc. (s. f.). JWT authentication plugin. Kong Docs. <https://developer.konghq.com/plugins/jwt/>
- Kong Inc. (s. f.). Prometheus plugin. Kong Docs. <https://developer.konghq.com/plugins/prometheus/>
- Kong Inc. (s. f.). Rate Limiting plugin. Kong Docs. <https://developer.konghq.com/plugins/rate-limiting/>
- Microsoft. (2026, 30 de abril). Configuration in ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/aspnet/core/fundamentals/configuration/>
- Microsoft. (2026, 30 de enero). OAuth 2.0 client credentials flow on the Microsoft identity platform. Microsoft Learn. <https://learn.microsoft.com/entra/identity-platform/v2-oauth2-client-creds-grant-flow>
- Microsoft. (s. f.). Add app roles to your application and receive them in the token. Microsoft Learn. <https://learn.microsoft.com/entra/identity-platform/howto-add-app-roles-in-apps>

OWASP Foundation. (2023). OWASP API Security Top 10 - 2023. <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

Prometheus Authors. (s. f.). Configuration. Prometheus documentation. <https://prometheus.io/docs/prometheus/latest/configuration/configuration/>

RabbitMQ. (s. f.). Consumer acknowledgements and publisher confirms. RabbitMQ documentation. <https://www.rabbitmq.com/docs/confirms>

RabbitMQ. (s. f.). Queues. RabbitMQ documentation. <https://www.rabbitmq.com/docs/queues>